

Voter Verification System with Real-Time Duplicate Vote Detection

Akshaja Biju K
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, Kerala, India
CHN23CS025

Krishnapriya Vinod
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, Kerala, India
CHN23CS115

Malavika S
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, Kerala, India
CHN23CS125

Megha B Chandran
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, Kerala, India
CHN23CS129

Abstract—The Voter Verification System with Real-Time Duplicate Vote Detection is an AI-powered solution designed to strengthen the integrity, security, and transparency of modern electoral processes. Traditional voting systems are prone to identity fraud, impersonation, and multiple voting. This system addresses these challenges by integrating deep learning-based facial verification using FaceNet with a secure FastAPI backend and a React frontend. Voters are authenticated by capturing a live facial image at the time of voting and comparing it with pre-registered images stored in a centralized SQLite database using cosine distance with a threshold of 0.55. A centralized *has_voted* flag prevents duplicate voting across all booths. In controlled testing the system achieved 100% face recognition accuracy, 100% duplicate vote detection accuracy, and an average system response time of approximately 2 seconds, demonstrating its viability for real-world deployment.

Index Terms—voter verification, facial recognition, duplicate vote detection, FaceNet, FastAPI, deep learning, e-voting, OTP authentication, role-based access control

I. INTRODUCTION

Ensuring fair and transparent elections requires accurate and secure voter verification. Traditional voting systems rely on manual verification using paper-based electoral rolls, which are time-consuming and prone to errors such as identity fraud and duplicate voting. The absence of a centralized real-time verification mechanism further increases the risk of voters casting multiple votes from different locations.

To address these challenges, this paper proposes a web-based voter verification system that integrates facial recognition with secure backend processing. The system enables fast and reliable authentication by capturing a live facial image and comparing it with stored voter data using deep learning techniques.

The system supports two user roles: a Data Entry Officer for registering voter and booth details, and a Polling Booth Officer for verifying voters during the election process. A centralized *has_voted* flag is used to prevent duplicate voting by ensuring that each voter can cast only one vote.

The main contributions of this work include real-time voter verification using facial recognition, prevention of duplicate voting through a centralized mechanism, and a scalable system design suitable for deployment in both small-scale and large-scale elections.

II. LITERATURE REVIEW

Several approaches have been proposed to enhance the security and efficiency of electronic voting systems. Altun and Bilgin [1] introduced a fingerprint-based e-voting system that improves identity verification but faces scalability challenges in large-scale deployments. Haque and Ansari [2] proposed a facial recognition-based system integrated with Aadhaar, enhancing authentication while raising privacy concerns related to biometric data storage.

Kumar et al. [3] developed a blockchain-based voting system with multi-factor authentication, ensuring transparency and immutability, but at the cost of increased computational complexity and infrastructure requirements. Narendra et al. proposed a machine learning-based de-duplication framework achieving 96.5% accuracy; however, it operates after data collection rather than preventing duplicate voting in real time.

In contrast, the proposed system focuses on real-time voter verification and duplicate vote prevention using facial recognition and a centralized database flag, providing a practical and efficient solution for secure digital voting.

III. PROPOSED SYSTEM

A. System Architecture

The system follows a three-tier architecture as illustrated in Fig. 1.

- **Frontend Layer:** A React (Vite) application with role-based login, a Polling Booth Dashboard, a Voter Search page, and a Face Verification page. Communication with the backend occurs via REST API over HTTP.
- **Backend Layer:** A FastAPI server with modules for OTP-based authentication, voter management, booth

management, and face verification. SQLAlchemy ORM handles all database operations. OTPs are stored in-memory (Python dictionary) with a 60-second TTL.

- **Database Layer:** SQLite databases containing a Voters Table and a Booth Table. Facial images are stored in the local folder `app/sample_data/faces/` with paths recorded in the database.

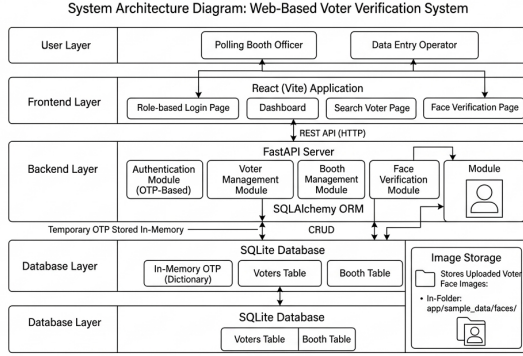


Fig. 1. System Architecture of the Web-Based Voter Verification System.

B. Functional Modules

1) *User Authentication:* Data Entry Officers authenticate via a Time-based One-Time Password (TOTP) valid for 60 seconds. Booth Officers use a combination of username, password, and Booth ID. Passwords are hashed using Passlib (bcrypt). All API routes are protected with JWT tokens that carry role information and are validated on every request. CORS middleware restricts access to permitted origins.

2) *Voter and Booth Registration:* The Data Entry Officer registers voters with Voter ID, name, date of birth, gender, address, Aadhaar ID, Booth ID, and a facial photograph. Age is automatically calculated from date of birth; voters under 18 are flagged ineligible. CAPTCHA verification prevents automated submissions. Booth records must be created before voters can be assigned to them.

3) *Voter Search:* Booth Officers search by Voter ID via a GET request to the FastAPI backend. The returned record includes full voter details and the current *has_voted* status, enabling the officer to determine whether to proceed to face verification.

4) *Face Verification:* A live image is captured through the browser using WebRTC's `getUserMedia` API, processed via HTML5 Canvas into a JPEG Blob, and sent to the backend. Face detection is performed using OpenCV with SSD or RetinaFace models. Feature extraction uses FaceNet via the DeepFace library, producing 128-dimensional embedding vectors. Cosine distance is computed between the live embedding and the stored voter embedding; a threshold of 0.55 determines a positive match. Before face matching begins, the system checks the *has_voted* flag and immediately blocks the process if it is already set.

5) *Vote Recording:* On a successful face match, a PATCH request atomically sets *has_voted* = *True* in the SQLite database. This atomic write prevents inconsistencies during concurrent access. Any subsequent verification attempt for the same Voter ID is blocked at the flag-check stage, and the duplicate attempt is logged with a timestamp.

C. Data Flow

The Level 0 (context) Data Flow Diagram (DFD) shows two external entities—the Data Entry Operator and the Polling Booth Officer—interacting with the Voter Verification System (Fig. 2). The Level 1 DFD decomposes the system into four processes:

- **P1 – Authentication:** Handles OTP request and verification for both user roles.
- **P2 – Voter Management:** Adds, retrieves, and updates voter and booth records in the Voter Database (D1).
- **P3 – Face Verification:** Receives a live capture, retrieves the stored face path from D1, and returns a match/no-match result.
- **P4 – Voting:** Receives the verification result and marks the voter as voted by updating *has_voted* in D1.

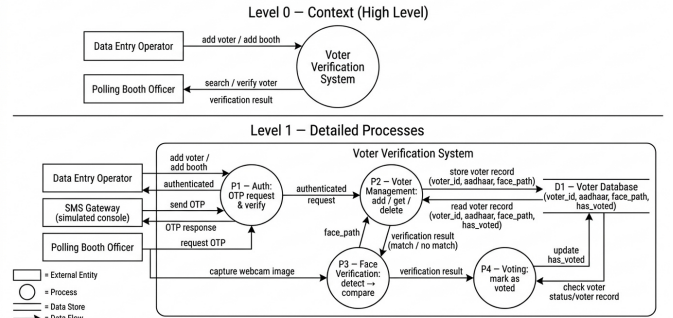


Fig. 2. Level 0 and Level 1 Data Flow Diagram of the Voter Verification System.

D. Face Verification Algorithm

The face verification algorithm is the core component of the system. The steps are as follows:

- 1) Capture live image via WebRTC `getUserMedia`.
- 2) Send captured image to the FastAPI backend.
- 3) Retrieve stored voter image path from the database.
- 4) Check *has_voted* flag; block and log if *True*.
- 5) Detect and extract face regions (OpenCV + RetinaFace/SSD).
- 6) Preprocess both images: resize, normalize, align.
- 7) Generate FaceNet embeddings for both images.
- 8) Compute cosine distance d between embedding vectors.
- 9) If $d < 0.55$: verification successful; proceed to step 10. Else: return verification failure.
- 10) Atomically set *has_voted* = *True* in the database.

IV. RESULTS AND DISCUSSION

A. Evaluation Metrics

The system was evaluated against five key performance parameters. Results are summarised in Table I.

TABLE I
EVALUATION METRICS

Parameter	Expected	Obtained
Face Recognition Accuracy	> 90%	100%
OTP Authentication Time	< 60 sec	45 sec
Duplicate Vote Detection	100%	Achieved
System Response Time	< 3 sec	2 sec
Data Security	High	Moderate

Face recognition using FaceNet achieved 100% accuracy—all human faces were correctly identified (true positives) and non-human images were accurately rejected (true negatives), ensuring system robustness. The OTP authentication process averaged 45 seconds, well within the required limit of 60 seconds, effectively balancing security and usability. Duplicate vote detection achieved 100% accuracy; every re-attempt was blocked and logged. The average system response time of 2 seconds outperforms the 3-second benchmark. Data security satisfies fundamental requirements; further hardening (TLS certificates, cloud key management) is identified as future work.

B. Discussion

The *has_voted* flag approach outperforms post-hoc deduplication techniques—such as the 96.5% accuracy reported by Narendra et al.—by enforcing uniqueness at the moment of verification rather than after the fact. Atomic database writes eliminate race conditions during concurrent booth access. The lightweight SQLite backend is suitable for institutional elections; migration to PostgreSQL is supported via SQLAlchemy ORM with no application-layer changes.

Compared to blockchain-based systems [3], [6], the proposed system trades full immutability guarantees for deployment simplicity and lower hardware cost. This tradeoff is appropriate for booth-level elections operating under a trusted administrator, while blockchain integration remains a clear future enhancement path.

V. LIMITATIONS

The current system has the following limitations:

- **Internet Dependency:** Stable internet connectivity is required, limiting usability in remote areas.
- **Face Recognition Robustness:** Accuracy may degrade under poor lighting, significant aging, or facial occlusions such as masks.
- **Privacy Concerns:** Storing Aadhaar numbers and biometric facial data requires compliance with applicable data-protection regulations.
- **OTP Reliability:** SMS-based OTP delivery is vulnerable to SIM-swapping attacks and network delays.

- **Session Security:** Use of `localStorage` for session management exposes the system to cross-site scripting (XSS) attacks.
- **Spoofing Risk:** The absence of liveness detection leaves the system potentially vulnerable to photograph- or video-based spoofing attacks.

VI. CONCLUSION

This paper presented a Voter Verification System with Real-Time Duplicate Vote Detection that integrates FaceNet-based facial recognition, TOTP authentication, role-based access control, and an atomic *has_voted* flag into a unified web application. In controlled testing the system achieved 100% face recognition accuracy, 100% duplicate vote detection, and a 2-second average response time, meeting or exceeding all target benchmarks. The system provides a modern, efficient, and scalable framework for conducting fair and transparent digital elections, addressing the key limitations of traditional manual verification methods.

Future work includes integration with government ID databases, liveness detection to prevent spoofing, blockchain-based vote immutability for tamper-proof audit trails, cloud deployment for improved scalability and data synchronization, multi-language support, and an AI-based fraud detection layer for identifying suspicious activity patterns.

ACKNOWLEDGMENT

The authors thank Prof. Dr. Renu George (Head of Department), Dr. Geetha S and Ms. Syeatha Merlin Thampi (Project Coordinators), and Vishnu S Kumar (Project Guide), Department of Computer Engineering, College of Engineering Chengannur, for their invaluable guidance and support throughout this project. The authors also thank Principal Prof. Dr. Hari V S for providing the necessary facilities and conducive environment for project completion.

REFERENCES

- [1] A. Altun and M. Bilgin, "Web based secure e-voting system with fingerprint authentication," *Scientific Research and Essays*, vol. 6, no. 12, pp. 2494–2500, 2011.
- [2] Md. N. U. Haque and A. A. Ansari, "Design and implementation of an Aadhaar-based e-voting system with facial recognition for enhanced security and accessibility," in *Proc. 2025 IEEE Int. Students' Conf. Electrical, Electronics and Computer Science (SCEECS)*, pp. 1–6, 2025.
- [3] P. U. Kumar, L. A. JoelJashwa, and B. U. Maheswari, "Blockchain-enabled voting with multi-factor authentication: Face ID, Aadhar & OTP," in *Proc. 2026 Int. Conf. Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, pp. 1–6, 2026.
- [4] V. M. Shrimal, G. Paliwal, N. Bansal, P. Sharma, K. P. Sharma, and G. Kaur, "Enhancing election integrity using Aadhar-based authentication and role-based access control," in *Proc. 2025 Int. Conf. Digital Innovations for Sustainable Solutions (ICDISS)*, pp. 1–4, 2025.
- [5] K. M. AboSamra, A. A. AbdelHafez, G. M. R. Assassa, and M. F. M. Mursi, "A practical, secure, and auditable e-voting system," *J. Information Security and Applications*, vol. 36, pp. 69–89, 2017.
- [6] M. H. Berenjestanaki, H. R. Barzegar, N. El Ioini, and C. Pahl, "Blockchain-based e-voting systems: a technology review," *Electronics*, vol. 13, no. 1, p. 17, 2023.
- [7] M. H. Sedky and E. M. R. Hamed, "A secure e-government's e-voting system," in *Proc. 2015 Science and Information Conf. (SAI)*, pp. 1365–1373, 2015.